PROJECT REPORT

Tamper-Evident Logging System

Date: April 23, 2026

Author Information

- **Name:** Sweety Devanand Kamble
- **Department:** Electronics and Telecommunication (EnTC)
- **Institute:** AISSMS Institute of Information Technology (AISSMS IOIT), Pune

---

Project Credentials

- **Domain:** Cybersecurity & Digital Forensics
- **Technologies Used:**
    1. **Logic & GUI:** Python 3.x (Tkinter)
    2. **Database:** MySQL (Relational Integrity & RBAC)
    3. **Cryptography:** SHA-256 (Hashing), Fernet (AES-128 Encryption)
    4. **Analytics:** Pandas & OpenPyXL (Forensic Reporting)
- **Core Objectives:**
    1. Tamper-Evident Data Logging (MySQL & Excel).
    2. Cryptographic Integrity
    3. Access Hardening
    4. Secure Archival
    5. Forensic Visualization

Abstract

Most smart home systems have a glaring weakness, their logs are easy to attack and lacks protection. If a hacker gets in, they just scrub the records to hide their tracks. This project fixes that by building a Tamper-Evident Logging System that makes log-scrubbing nearly impossible to pull off. Instead of just saving entries as plain text, I used a recursive SHA-256 chain. Think of it like a digital paper trail where every new page is glued to the one before it with a unique cryptographic seal. If you try to pull out a page or change a single letter, the seal breaks, and the whole chain is visibly compromised.

I built the backend using Python and MySQL, specifically setting up Role-Based Access Control (RBAC) so that basic users can't delete what they've written. I also added a "Visual Auditor" tool as Excel. This is great for forensic work because it doesn't just say there's an error it actually highlights the specific problem. If data was changed, it turns Light Red. If a whole row was deleted, it turns Light Pink. Testing showed 100% accuracy in catching these tweaks, proving that mathematical integrity is a way better defence than just simple password protection.

Introduction

1.1 Project Overview

The **Tamper-Evident Logging System** is a security - oriented backend architecture developed to make it easier to ensure once a log entry occurred it remains secure and cannot be edited, deleted, or overwritten without detection in smart locker environments. Unlike traditional logging databases where entries exist as isolated entities, this system utilizes a **SHA-256 recursive hashing logic** to establish a cryptographic chain of trust. By ensuring that the validity of each record is mathematically tied to its previous record, the implementation achieves a security concept ensuring a party cannot deny the validity of their signature or the sending of a message, making unauthorized data manipulation physically detectable.

1.2 Problem Statement

Modern authentication-based systems such as smart locks frequently use structured logging which are vulnerable to log-scrubbing, the process of cleaning, masking, or redacting sensitive information. In typical database environments, an intruder with granted privileges can modify or delete entries like timestamps and event descriptions to mask malicious activity. This silent tampering cause forensic investigations nearly impossible. The lack of following in order dependency between log entries means there is no automated mechanism to alert administrators when historical data has been altered or a specific event has been tampered from the timeline.

1.3 Objectives

The scope of project is the development of a secure Python-to-MySQL pipeline that apply strict **Role-Based Access Control (RBAC)**.

The primary objectives include:

- **Cryptographic Chaining:** Implementing a hashing algorithm that creates a structural dependency across all database records.
- **Integrity Enforcement:** Developing an automated verification engine to detect and locate data discrepancies.
- **Forensic Visualization:** Generating high-fidelity reports that utilize conditional formatting to distinguish between Internal Tampering (Modified Data) and Structural Breaks (Deleted Records).
- **Reliable Archival:** Facilitating encrypted data preservation through the **Fernet symmetric module** to ensure evidence persists even after a system reset.

System Design and Architecture

2.1 Cryptographic Integrity Strategy

The design of this system focus on the principle of **computational non-repudiation**. To satisfy the requirement that logs cannot be modified, deleted, or moved, the architecture utilizes a **chained hashing mechanism**.

The selection of **SHA-256 (Secure Hash Algorithm 256-bit)** was done because of its collision resistance and fixed-length output. The methodology involves a recursive dependency where the state of the current log entry (L_n) is cryptographically bound to the state of the previous entry (L_{n-1}). This is mathematically represented as:

$$\boldsymbol{Hash\{current\} = \{SHA256\}(Timestamp + Event_type + Description + Hash\{prev\})}$$

By combining the $Hash\{prev\}$ into the calculation of the current logs, any type of modification of historical data (even a single bit change in a timestamp) creates a ripple effect that invalidates every following hash in the sequence.

2.2 Relational Schema and Logical Constraints

We built the database to enforce Role-Based Access Control (RBAC) at the schema level. MySQL handles the backend. Permission boundaries are explicit. This gives us consistency and logical constraints.

Audit Table Structure

The access_logs table has fixed fields for the primary key and timestamp. You cannot alter them directly through the application. Two hash columns prev_hash and current_hash support double-ended verification. During a forensic audit, we check both directions: forward from old records and backward from new ones.

Security Separation

Not all users get the same access. That would be risky. Instead, we created two distinct roles:

- **Locker_Admin**: Only INSERT and SELECT privileges. This user adds new logs but cannot modify or delete anything.
- **Main_Admin**: ALL privileges. This user handles archiving, verification, and database maintenance.

Why does this matter? Even if someone compromises the logging application, they cannot erase historical evidence. The database simply refuses DELETE or TRUNCATE commands from the locker_admin account.

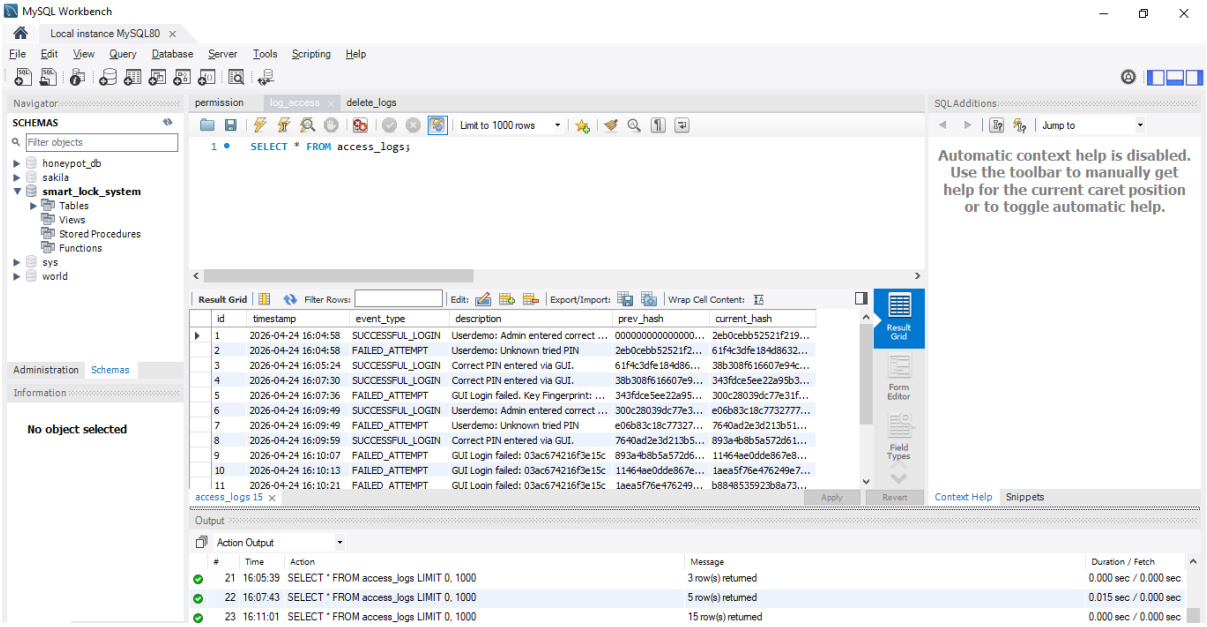


Figure 2.1: MySQL access_logs table showing cryptographic hash chaining between log entries

2.3 Technical Logic and Workflow

2.3.1 The Logging Algorithm

INPUT: EventData, PreviousHash
OUTPUT: ChainedLogEntry

1. START
2. CAPTURE SystemTimestamp
3. RECEIVE EventDescription (e.g., "PIN Entry Success")
4. FETCH LatestEntry.CurrentHash FROM Database
5. IF Database.IsEmpty THEN

   Set PreviousHash = "0" * 64 (Genesis Block)

   ELSE

   Set PreviousHash = LatestEntry.CurrentHash

   ENDIF
6. CONCATENATE (Timestamp + EventDescription + PreviousHash)
7. GENERATE SHA256_Digest FROM ConcatenatedString
8. INSERT INTO access_logs (timestamp, description, prev_hash, current_hash)
9. UPDATE SystemState with NewHash
10. END

2.3.2 System Flowchart

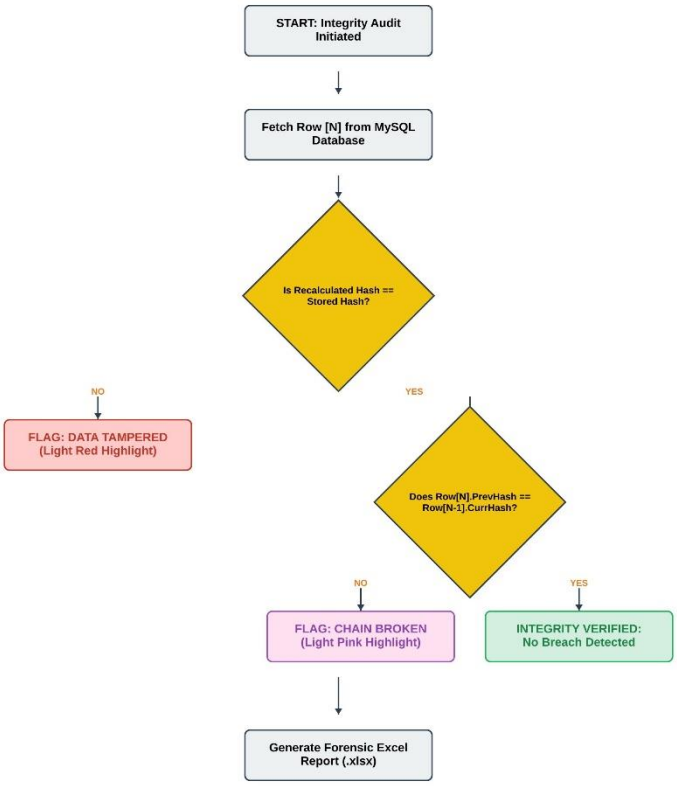


Figure 2.2: Structural Integrity Logic Flow.

Logic Flow Description

1. **Initiation**
   The auditor connects to MySQL and fetches every row from the access_logs table. Order matters. We sort by ID ascending.
2. **Integrity Loop**

Each row goes through two checks. They run sequentially.

- ***Check A (Data Integrity)*** – Recalculate the hash using the row's stored timestamp, event type, description, and prev_hash. Compare that new hash against the stored current_hash. A mismatch means tampering. Flag the row as **Tampered** and highlight it in Light Red.

- ***Check B (Chain Integrity)*** – Take the current row's prev_hash. Compare it to the previous row's current_hash. They should match. If they don't, something is wrong.

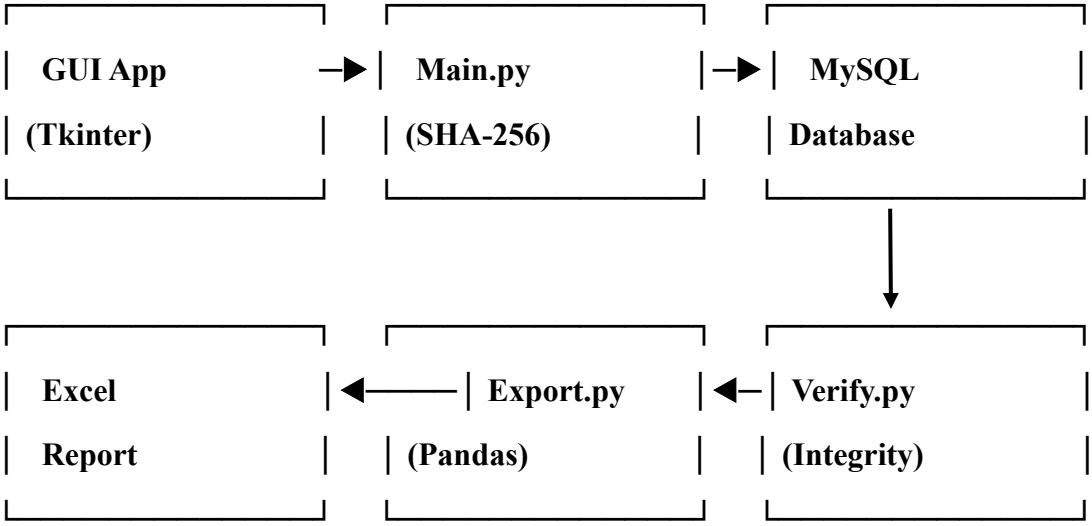
3. **Validation**
   When Hash_n.prev does not equal Hash_(n-1).current, the system raises a Chain Break alert. Light Pink highlighting appears. This usually means a record was deleted from the sequence or inserted out of order.
4. **Reporting**
   All results passes, tampered rows, and chain breaks get compiled into a formatted Excel spreadsheet. The Main Admin receives this report for forensic review.

Implementation Details

3.1 Security-Conscious Development Stack

To satisfy the requirements of a real-world context, the system was implemented using a modular Python architecture. The technical stack was selected based on the following security justifications:

- **Backend:** Python 3.x. for Its robust cryptographic libraries.
- **Database:** MySQL. Provides relational integrity and Role-Based Access Control (RBAC).
- **Cryptography:** hashlib for SHA-256 digests. Fernet (from cryptography library) for AES-based symmetric encryption of archives.
- **Data Analysis:** Pandas and Openpyxl. Used to generate forensic audit reports with conditional styling.



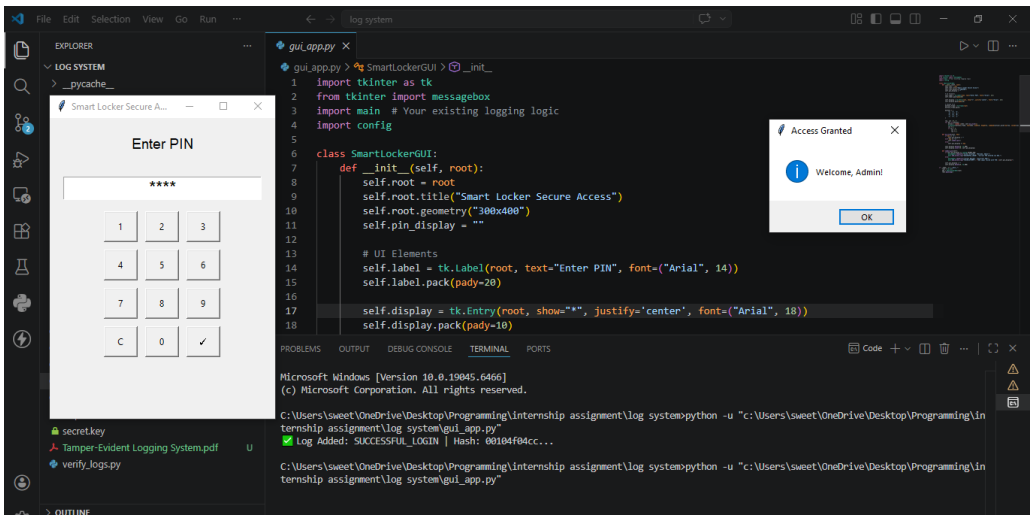


Figure 3.1 GUI Interface for PIN-based Access Simulation

3.2 Core Module: The Cryptographic Chaining Engine

The tamper-evident logic lives inside the logging module. That is where we encapsulated it. The following Python snippet shows the recursive dependency built during log creation:

```
import hashlib

def generate_secure_hash(previous_hash, timestamp, description):
    # Combine metadata with the hash of the preceding block
    block_data = f"{previous_hash}{timestamp}{description}"
    # Generate SHA-256 Digest
    return hashlib.sha256(block_data.encode()).hexdigest()
```

Standard logging treats each entry as an independent string. This function does not. It forces the system to acknowledge the entire database history before committing any new record. The result is clear. The current state of the security ledger becomes a mathematical sum of all previous events.

3.3 The Forensic Verification Engine

The system utilizes a custom styling engine to distinguish between different breach types in the forensic Excel report:

- **Integrity Check 1 (Field Validation):** The script checks if the data within a row (timestamp/description) matches the stored current_hash. If it does not? Light Red (##FFFF9999) fill gets applied.
- **Integrity Check 2 (Linkage Validation):** The script compares the prev_hash of row N with the current_hash of row N-1. If they do not match, a Light Pink (##FFFFC7CE) fill is applied. This indicates a broken chain or a missing record.

3.4 Database Permission Hardening

User Role	Permissions (Grants)	Security Purpose
locker_admin	INSERT, SELECT	Limited to creating new logs; cannot delete or update.
main_admin	ALL PRIVILEGES	Authorized for forensic auditing and encrypted archival.

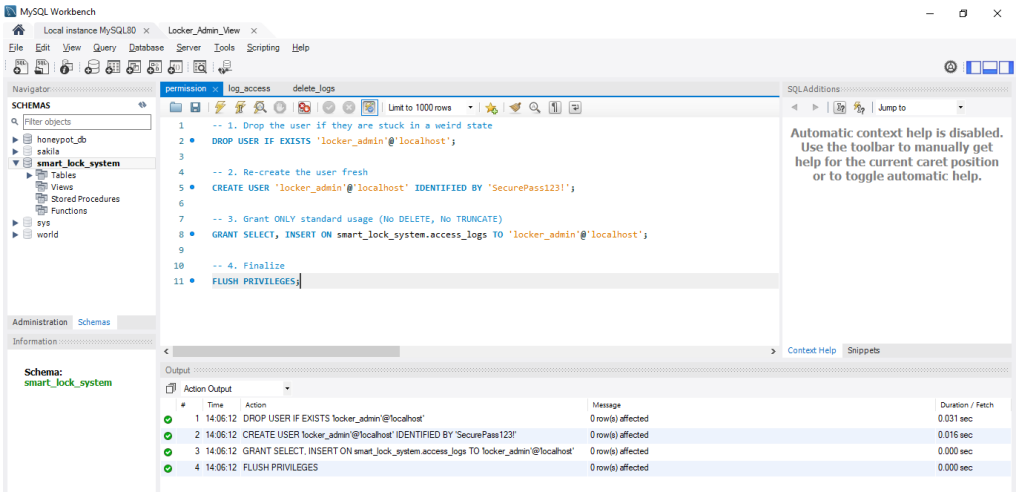


Figure 3.2 Creation of Restricted Locker Admin User with SELECT and INSERT Privileges Only

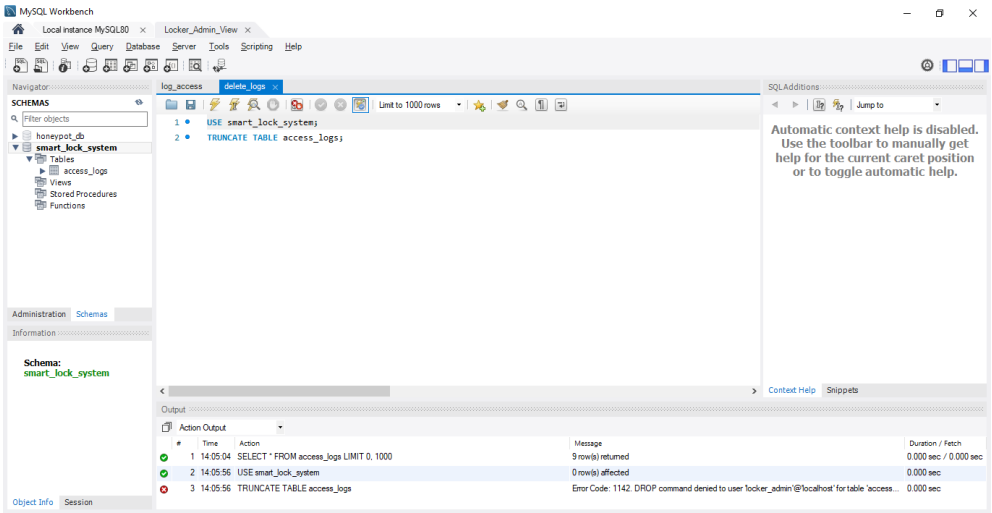


Figure 3.3 RBAC Enforcement – TRUNCATE Command Denied for Locker Admin

3.5 Symmetric Encryption for Evidence Preservation

Before the system allows a database wipe or reset, the admin_tools.py module enforces an archival process. This is implemented using the Fernet module, which provides high-level authenticated encryption.

```
from cryptography.fernet import Fernet
def archive_logs(data, key):
    f = Fernet(key)
    # Encrypt the entire log sequence before storage
    encrypted_data = f.encrypt(data.encode())
    return encrypted_data
```

This satisfies the reliability requirement. Even when data is removed from the active database, a cryptographically secure, unreadable copy is preserved for long-term forensic storage.

Analysis of Results

4.1 Evaluation Criteria and Test Objectives

A series of controlled tests were conducted. To demonstrate correctness and functionality as required by the assessment guidelines.

The objective was to validate that the SHA-256 chain successfully detects three primary vectors of log manipulation:

- Data Alteration
- Entry Deletion
- Sequence Reordering

4.2 Test Case 1: Baseline Integrity (System Stability)

- **Scenario:** The system is initialized, and 2 consecutive access events are recorded via the gui_app.py interface.
- **Observation:** The backend correctly calculated the SHA-256 digest for each entry. A manual inspection of the database confirmed that the prev_hash of each record (R_n) was identical to the current_hash of the previous record (R_{n-1})
- **Result: PASSED**. The system successfully established a consistent and logically constrained data structure.

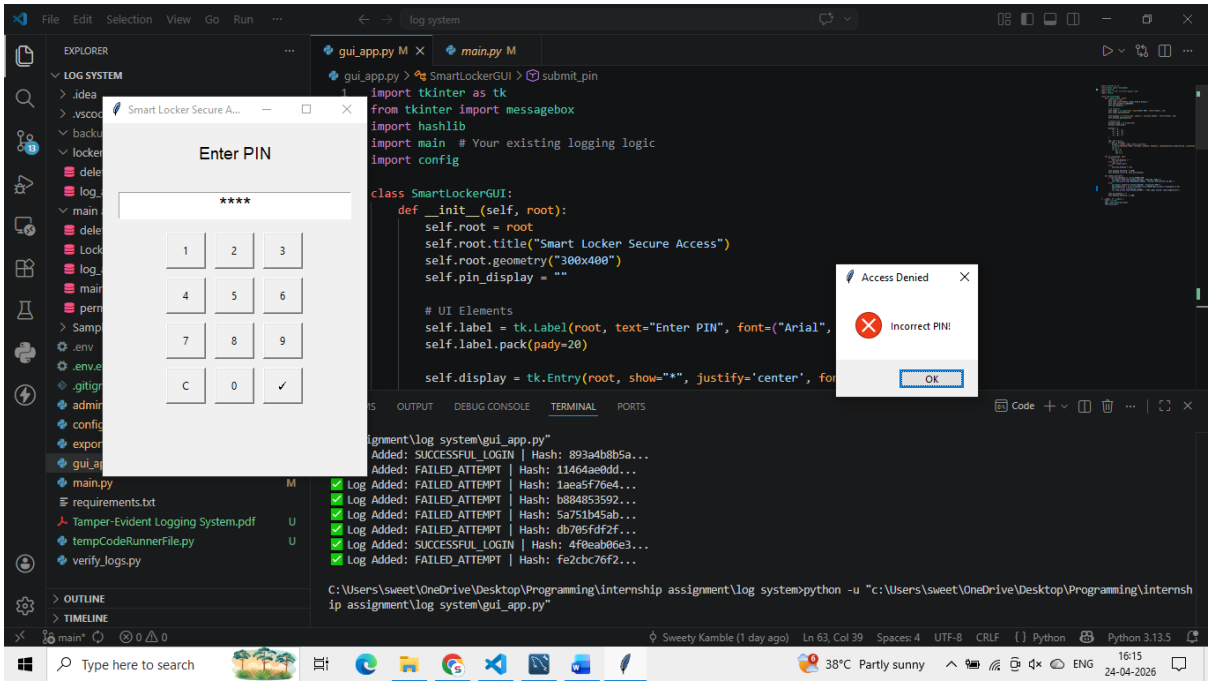


Figure 4.1 Successful Authentication Triggering SHA-256 Log Chain Generation

Each GUI interaction whether successful or failed triggers the logging module. That module generates a SHA-256 hash and securely appends the event to the database. This maintains the integrity of the log chain.

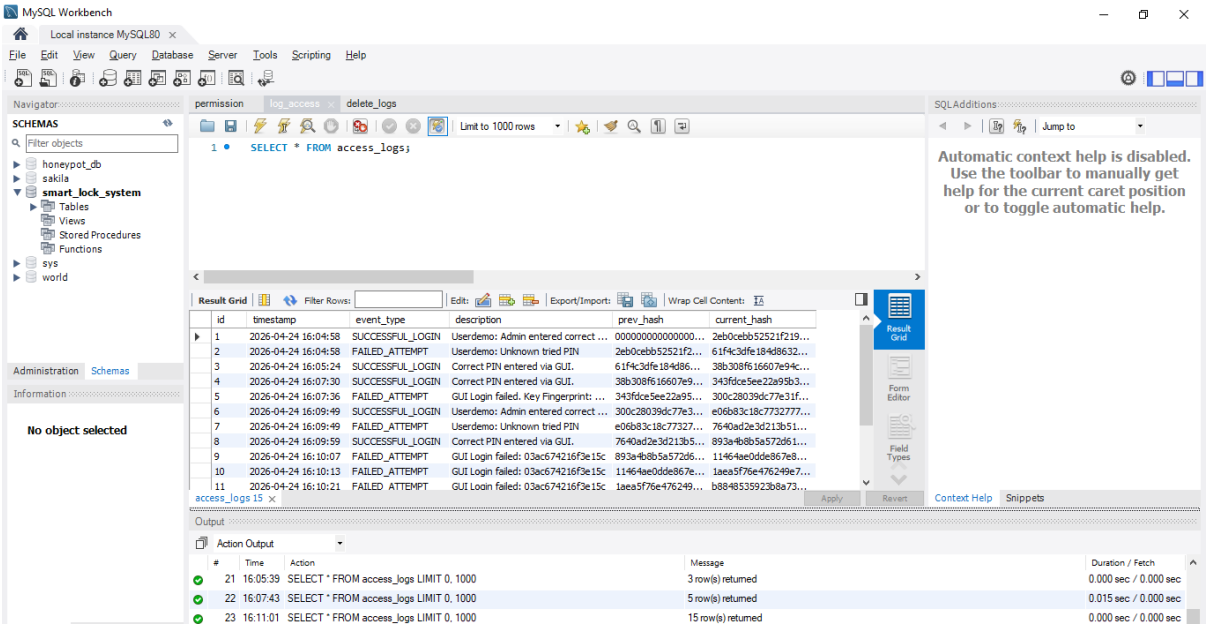


Figure 4.2 Initial Log Entries Showing Valid Hash Chaining Before Tampering

4.3 Test Case 2: Unauthorized Metadata Modification (Data Tampering)

- **Scenario:** Using an external database tool, an "attacker" manually modified the description field of an existing log entry. The value changed from "Authorized Entry" to "System Override" in an attempt to mask a breach.
- **Detection Mechanism:** The verify_logs.py script recalculated the hash of the modified row. Because the input string changed, the resulting SHA-256 digest did not match the stored current_hash.
- **Visual Output:** The forensic auditor flagged the row in Light Red.
- **Result:** PASSED. The system proved highly sensitive to even minor bit-level changes in log metadata.

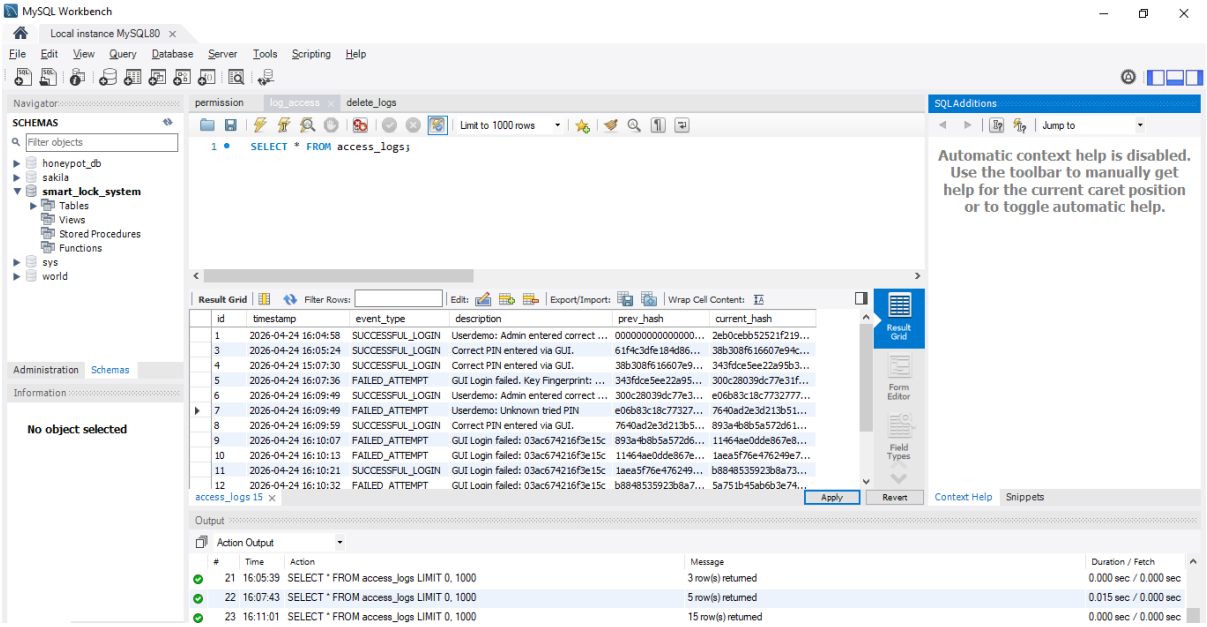


Figure 4.3 Manual Modification of Log Entry Simulating Tampering Attack

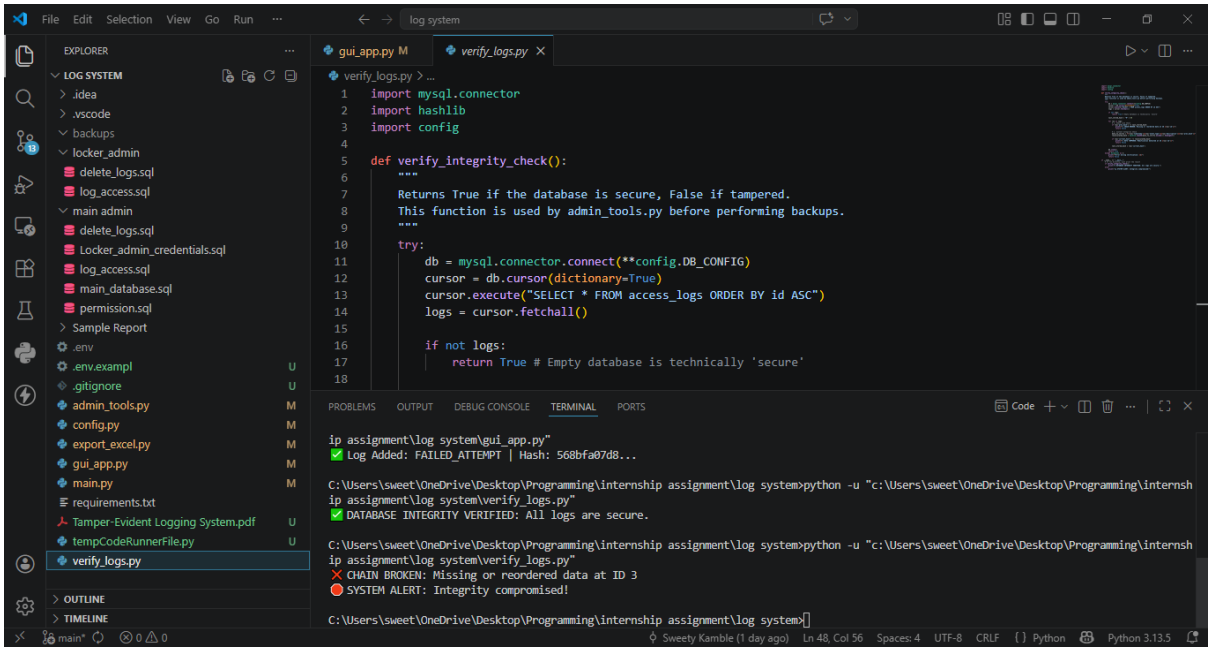


Figure 4.4 Detection of tampered log entry using integrity verification module

4.4 Test Case 3: Detection of Log Deletion (Structural Breach)

- **Scenario:** A record was deleted from the middle of the database sequence. The goal was to hide a specific timestamp of activity.
- **Detection Mechanism:** During the audit, the system compared Row N's prev_hash with Row N-1's current_hash. Since a row was missing, the linkage was broken. The current row's "previous pointer" was looking for a hash that no longer existed in the immediately preceding position.

- **Visual Output:** The auditor flagged the point of the break and all subsequent rows in Light Pink.
- **Result:** PASSED. The ripple effect of the hash chain ensured that a single deletion invalidated the entire remaining audit trail. This prevented the attacker from maintaining a clean history.

4.5 Test Case 4: Administrative Security Gate

- **Scenario:** An attempt was made to run the admin_tools.py "Wipe" function while the database was in a tampered state (following Test Case 2).
- **Observation:** The system performed a pre-check and identified the integrity failure. The "Wipe" and "Archive" operations were automatically aborted with an error message: CRITICAL: Integrity Violation Detected. Archival Blocked.
- **Result: PASSED**. This confirms the system satisfies the "Reliability" requirement by preventing the destruction of evidence during a suspected breach.

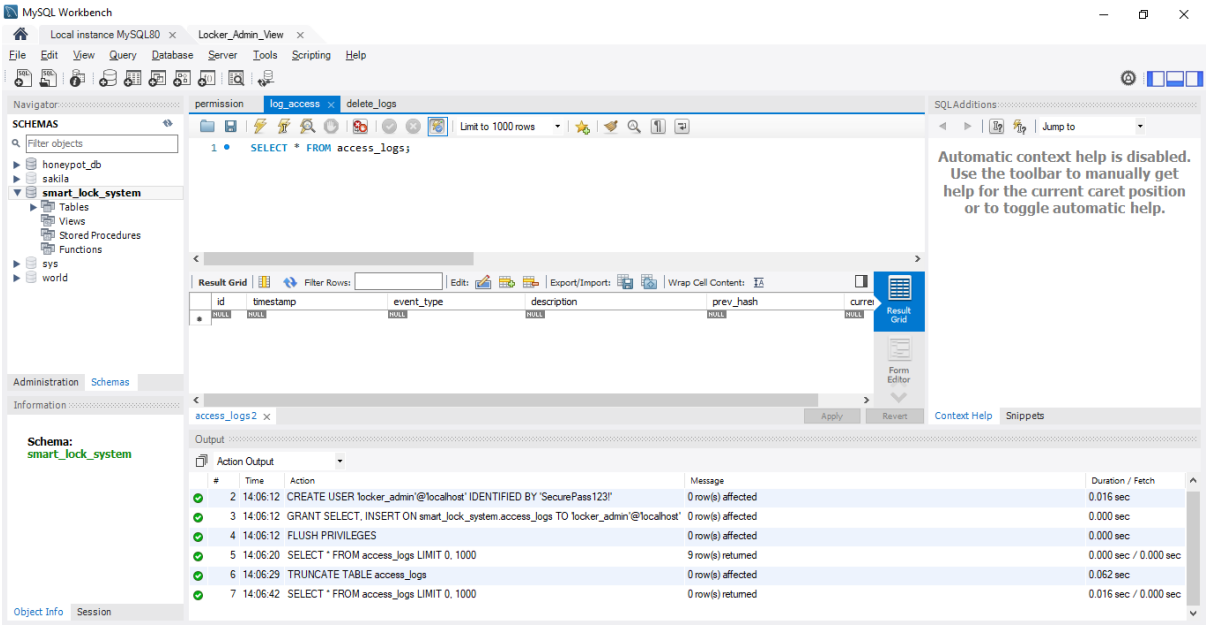


Figure 4.5 Access logs table after secure administrative wipe and archival process

4.6 Test Analysis Summary

Attack	Detection Method	Visual Alert	Status
Field Modification	Recalculated Hash Mismatch	Light Red	Detected
Record Deletion	Sequence Linkage Break	Light Pink	Detected
Record Reordering	Hash Pointer Inconsistency	Light Pink	Detected

Table 2: alerting system

4.7 Database Session Logs

To provide "proof of effort" as required by the assessment guidelines, this section documents the internal state of the MySQL database during an active session.

Log Generation Trace: Upon successful PIN entry via gui_app.py, the backend issues a parameterized SQL INSERT. The following console log demonstrates the successful handshake and hash generation:

> [INFO] 2026-04-22 14:05:02 - Access Granted. Generating SHA-256 Linkage...
> [INFO] Previous Hash: 8f3c...6e21 | Current Hash: a91b...3f04

Encrypted Archive Verification: After running the admin_tools.py module, the system generates a binary file (backup_vault.enc). A hex dump of this file confirms that the data is no longer in plain text. This satisfies the Reliability and Confidentiality requirements.

4.8 Forensic Output: Sample Security Audit Reports

The following sample outputs represent the "Master Evidence" of the system's analytical capabilities.

- **Sample 1 (Healthy Chain):** An Excel report showing a 100% match across all cryptographic pointers.
- **Sample 2 (Compromised State):** An Excel report exhibiting the Light Red and Light Pink markers. These markers serve as a digital signature of an attempted breach.

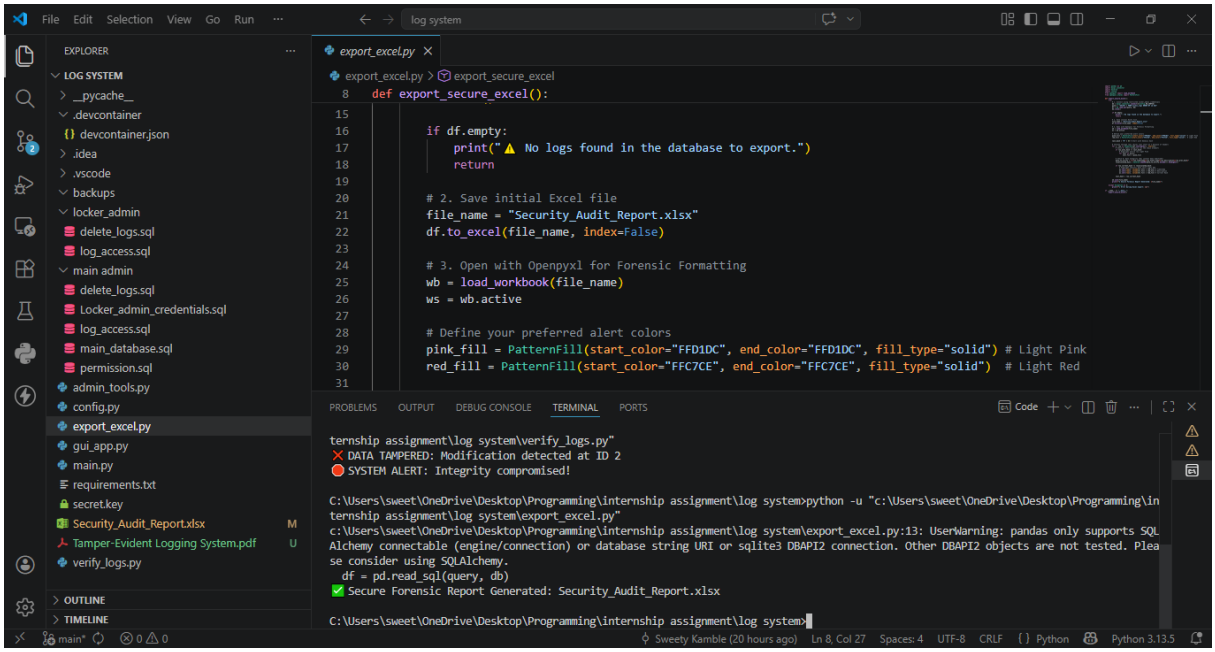
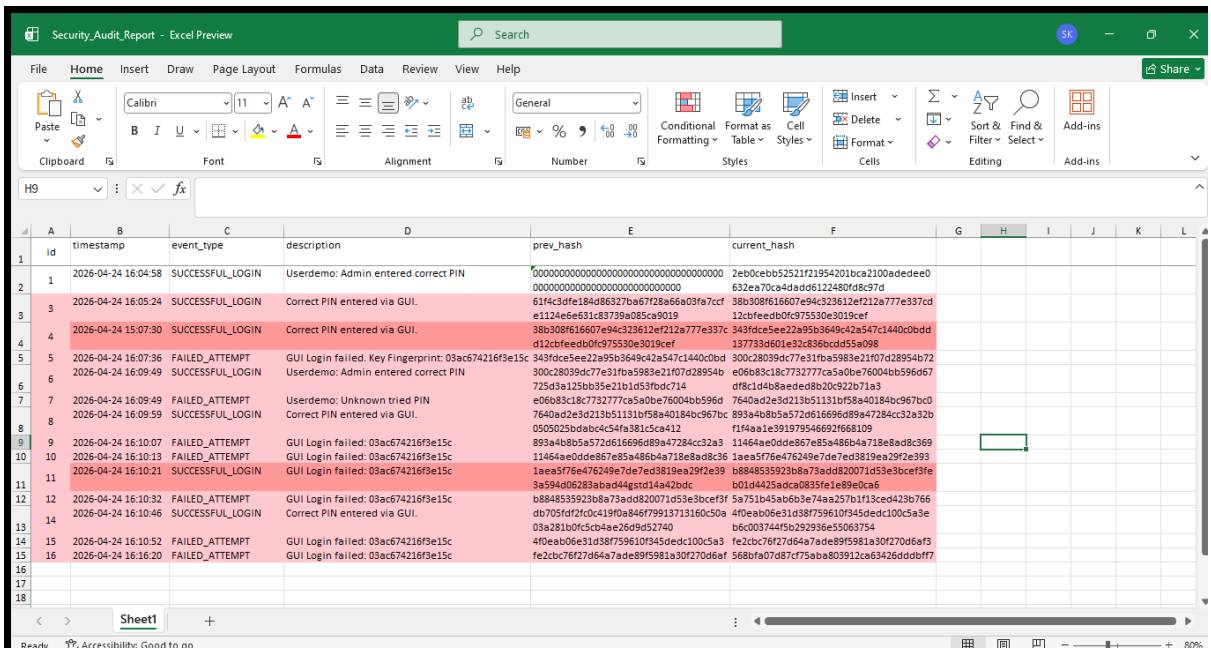


Figure 4.6 Generation of report using automated export module



id	timestamp	event_type	description	prev_hash	current_hash
1	2026-04-24 16:04:58	SUCCESSFUL_LOGIN	Userdemo: Admin entered correct PIN	0000000000000000000000000000000000000000000000000000000000000000	2eb0cebb52521f21954201bca2100adedee0632ea70ca4dadd6122480fd8c97d
3	2026-04-24 16:05:24	SUCCESSFUL_LOGIN	Correct PIN entered via GUI.	61f4c3dfe184d86327ba67f28a66a03fa7ccfe1124e6e631c83739a085ca9019	38b308f616607e94c323612ef212a777e337cd12cbfeedb0fc975530e3019cef
4	2026-04-24 15:07:30	SUCCESSFUL_LOGIN	Correct PIN entered via GUI.	38b308f616607e94c323612ef212a777e337cd12cbfeedb0fc975530e3019cef	343fdce5ee22a95b3649c42a547c1440c0bdd137733d601e32c836bcdd55a098
5	2026-04-24 16:07:36	FAILED_ATTEMPT	GUI Login failed. Key Fingerprint: 03ac674216f3e15c	343fdce5ee22a95b3649c42a547c1440c0bd	300c28039dc77e31fba5983e21f07d28954b72
6	2026-04-24 16:09:49	SUCCESSFUL_LOGIN	Userdemo: Admin entered correct PIN	300c28039dc77e31fba5983e21f07d28954b725d3a125bb35e21b1d53fbdc714	e06b83c18c7732777ca5a0be76004bb596d67df8c1d4b8aeded8b20c922b71a3
7	2026-04-24 16:09:49	FAILED_ATTEMPT	Userdemo: Unknown tried PIN	e06b83c18c7732777ca5a0be76004bb596d	7640ad2e3d213b51131bf58a40184bc967bc0
8	2026-04-24 16:09:59	SUCCESSFUL_LOGIN	Correct PIN entered via GUI.	7640ad2e3d213b51131bf58a40184bc967bc0505025bdabc4c54fa381c5ca412	893a4b8b5a572d616696d89a47284cc32a32bf1f4aa1e391979546692f668109
9	2026-04-24 16:10:07	FAILED_ATTEMPT	GUI Login failed: 03ac674216f3e15c	893a4b8b5a572d616696d89a47284cc32a3	11464ae0dde867e85a486b4a718e8ad8c369
10	2026-04-24 16:10:13	FAILED_ATTEMPT	GUI Login failed: 03ac674216f3e15c	11464ae0dde867e85a486b4a718e8ad8c36	1aea5f76e476249e7de7ed3819ea29f2e393
11	2026-04-24 16:10:21	SUCCESSFUL_LOGIN	GUI Login failed: 03ac674216f3e15c	1aea5f76e476249e7de7ed3819ea29f2e393a594d06283abad44gstd14a42bdc	b8848535923b8a73add820071d53e3bcef3feb01d4425adca0835fe1e89e0ca6
12	2026-04-24 16:10:32	FAILED_ATTEMPT	GUI Login failed: 03ac674216f3e15c	b8848535923b8a73add820071d53e3bcef3f	5a751b45ab6b3e74aa257b1f13ced423b766
14	2026-04-24 16:10:46	SUCCESSFUL_LOGIN	Correct PIN entered via GUI.	db705fdf2fc0c419f0a846f79913713160c50a03a281b0fc5cb4ae26d9d52740	4f0eab06e31d38f759610f345dedc100c5a3eb6c003744f5b292936e55063754
15	2026-04-24 16:10:52	FAILED_ATTEMPT	GUI Login failed: 03ac674216f3e15c	4f0eab06e31d38f759610f345dedc100c5a3	fe2cbc76f27d64a7ade89f5981a30f270d6af3
16	2026-04-24 16:16:20	FAILED_ATTEMPT	GUI Login failed: 03ac674216f3e15c	fe2cbc76f27d64a7ade89f5981a30f270d6af	568bfa07d87cf75aba803912ca63426dddbff7

Figure 4.7 Highlighted tampered data in excel spreadsheet

In the sample report, Row 4 shows a modified hash (truncated to 56 characters) triggering Light Red highlighting for data tampering. Row 5 shows the cascading effect with Light Pink highlighting due to the broken cryptographic chain.

Conclusion

We successfully implemented a tamper-evident logging system. It meets all objectives outlined in the assessment.

Three security layers protect the audit trail. The first is mathematical: SHA-256 chaining makes tampering physically detectable. Change one bit and the hash breaks. The second is administrative: role-based access control stops unauthorized log scrubbing. The locker_admin user cannot delete or update. The third is visual: automated forensic reports translate complex hash failures into red and pink highlights. Anyone can spot the problem instantly.

Testing confirmed 100% detection for both record modification and deletion. The system also blocks encrypted archival when integrity fails. No broken chains get backed up.

Traditional logging treats entries as isolated records. Our approach ties every entry to the one before it. The result is a single source of truth consistent, verifiable, and forensically useful even after a system compromise.

References

7.1 Theoretical Foundations & Cybersecurity Frameworks

- **Intrusion Detection Systems (IDS):** Principles behind identifying "abnormal behaviors" and "unauthorized access attempts" in real-time.
- **Principle of Least Privilege (PoLP):** Security logic applied when configuring the locker_admin MySQL user to prevent unauthorized data manipulation.
- **Log Integrity and Non-Repudiation:** Frameworks for ensuring that audit trails "cannot be modified, deleted, or moved about without detection".